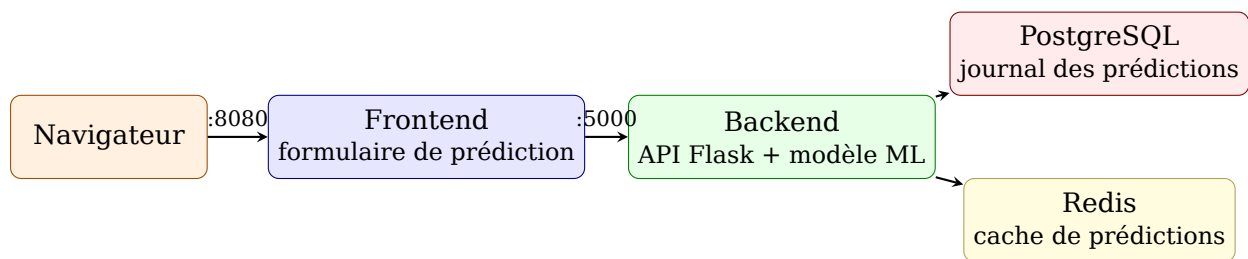


Mise en situation

MTU Corp élargit son activité : en plus de sa propre flotte, elle propose désormais la location de véhicules à des chauffeurs-livreurs partenaires indépendants. Avant d'accorder une location, elle veut évaluer automatiquement la solvabilité du partenaire à partir de quelques indicateurs simples : revenu annuel, nombre de prêts en cours, jours de retard de paiement cumulés, et historique de crédit qualitatif.



Fichiers fournis

data.csv

revenu_annuel	nombre_prets	jours_retard_paiement	historique_credit	score_credit
45000	1	2	Bon	Eleve
32000	3	15	Standard	Standard
28000	4	25	Mauvais	Faible
52000	0	0	Bon	Eleve
39000	2	10	Standard	Standard
25000	5	30	Mauvais	Faible
61000	1	1	Bon	Eleve
34000	3	12	Standard	Standard
22000	6	35	Mauvais	Faible
47000	1	3	Bon	Eleve
30000	2	18	Standard	Standard
19000	7	40	Mauvais	Faible
55000	0	0	Bon	Eleve
36000	2	9	Standard	Standard
24000	5	28	Mauvais	Faible
43000	1	4	Bon	Eleve
33000	3	14	Standard	Standard
21000	6	33	Mauvais	Faible
58000	0	1	Bon	Eleve
31000	2	16	Standard	Standard
23000	5	29	Mauvais	Faible
49000	1	2	Bon	Eleve

37000,2,11,Standard,Standard
20000,7,38,Mauvais,Faible

backend/train_model.py

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import joblib
import json
```

```
df = pd.read_csv("data.csv")
```

```
encodeur_historique = LabelEncoder()
df["historique_enc"] = encodeur_historique.fit_transform(df["historique_credit"])
```

```
X = df[["revenu_annuel", "nombre_prets", "jours_retard_paiement", "historique_enc"]]
y = df["score_credit"]
```

```
# Division en ensemble d'entrainement (80%) et de test (20%)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

```
modele = DecisionTreeClassifier(random_state=0)
modele.fit(X_train, y_train)
```

```
y_pred = modele.predict(X_test)
metriques = {
    "accuracy": accuracy_score(y_test, y_pred),
    "precision": precision_score(y_test, y_pred, average="macro", zero_division=0),
    "recall": recall_score(y_test, y_pred, average="macro", zero_division=0),
    "f1": f1_score(y_test, y_pred, average="macro", zero_division=0),
    "taille_train": len(X_train),
    "taille_test": len(X_test),
}
```

```
joblib.dump({"modele": modele, "encodeur": encodeur_historique}, "data/model.joblib")
with open("data/metrics.json", "w") as f:
    json.dump(metriques, f, indent=2)
```

```
print("Modele entraine.")
print(metriques)
```

backend/app.py

```
from flask import Flask, jsonify, request
from flask_cors import CORS
import os, json, time
import joblib
import psycpg2
import redis
```

```

app = Flask(__name__)
CORS(app)

modele_charge = joblib.load("data/model.joblib")

def get_db():
    return psycopg2.connect(
        host=os.environ.get("DB_HOST", "db"),
        dbname=os.environ.get("DB_NAME", "mtudb"),
        user=os.environ.get("DB_USER", "mtu"),
        password=os.environ.get("DB_PASSWORD", ""))

def get_cache():
    return redis.Redis(host=os.environ.get("CACHE_HOST", "cache"),
                       port=6379, decode_responses=True)

def enregistrer(revenu, prets, retard, historique, prediction, source):
    conn = get_db()
    cur = conn.cursor()
    cur.execute(
        "INSERT INTO predictions (revenu_annuel, nombre_prets, jours_retard, "
        "historique_credit, score_predit, source) VALUES (%s,%s,%s,%s,%s,%s)",
        (revenu, prets, retard, historique, prediction, source))
    conn.commit()
    conn.close()

@app.route("/api/predire")
def predire():
    revenu = float(request.args.get("revenu"))
    prets = int(request.args.get("prets"))
    retard = int(request.args.get("retard"))
    historique = request.args.get("historique")

    cache = get_cache()
    cle = f"prediction:{revenu}:{prets}:{retard}:{historique}"
    resultat_cache = cache.get(cle)
    if resultat_cache:
        data = json.loads(resultat_cache)
        data["source"] = "cache"
        enregistrer(revenu, prets, retard, historique, data["score_predit"], "cache")
        return jsonify(data)

    time.sleep(1) # simule un modele plus lourd en production
    historique_enc = modele_charge["encodeur"].transform([historique])[0]
    prediction = modele_charge["modele"].predict([[revenu, prets, retard, historique_enc]])

    resultat = {"revenu": revenu, "prets": prets, "retard": retard,
               "historique": historique, "score_predit": prediction, "source": "calcul"}
    cache.setex(cle, 60, json.dumps(resultat))
    enregistrer(revenu, prets, retard, historique, prediction, "calcul")
    return jsonify(resultat)

```

```

@app.route("/api/performance")
def performance():
    with open("data/metrics.json") as f:
        return jsonify(json.load(f))

@app.route("/api/historique")
def historique():
    conn = get_db()
    cur = conn.cursor()
    cur.execute("SELECT revenu_annuel, nombre_prets, jours_retard, historique_cr
                \"score_predit, source FROM predictions ORDER BY id DESC LIMIT 20")
    rows = cur.fetchall()
    conn.close()
    return jsonify([{"revenu": r[0], "prets": r[1], "retard": r[2],
                    "historique": r[3], "score_predit": r[4], "source": r[5]}
                    for r in rows])

@app.route("/health")
def health():
    return jsonify(status="ok")

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)

backend/requirements.txt
flask==3.0.3
flask-cors==4.0.1
psycopg2-binary==2.9.9
redis==5.0.4
scikit-learn==1.5.0
pandas==2.2.2
joblib==1.4.2

db/init.sql
CREATE TABLE predictions (
    id SERIAL PRIMARY KEY,
    revenu_annuel NUMERIC NOT NULL,
    nombre_prets INTEGER NOT NULL,
    jours_retard INTEGER NOT NULL,
    historique_credit TEXT NOT NULL,
    score_predit TEXT NOT NULL,
    source TEXT NOT NULL,
    cree_le TIMESTAMP DEFAULT NOW()
);

frontend/index.html
<!DOCTYPE html>
<html>
<head><title>MIU Corp – Solvabilite partenaire</title></head>
<body>

```

```

<h1>MIU Corp – Evaluation de solvabilite </h1>
<label>Revenu annuel : <input type="number" id="revenu" value="35000"></label>
<label>Nombre de prets en cours : <input type="number" id="prets" value="2"></label>
<label>Jours de retard cumules : <input type="number" id="retard" value="12"></label>
<label>Historique de credit :
  <select id="historique">
    <option value="Mauvais">Mauvais</option>
    <option value="Standard">Standard</option>
    <option value="Bon">Bon</option>
  </select>
</label><br>
<button onclick="predire()">Evaluer</button>
<p id="resultat"></p>
<h2>Historique</h2>
<button onclick="chargerHistorique()">Voir l'historique</button>
<ul id="historiqueListe"></ul>
<script src="app.js"></script>
</body>
</html>

```

frontend/app.js

```

const API_BASE = "http://localhost:5000";

async function predire() {
  const revenu = document.getElementById("revenu").value;
  const prets = document.getElementById("prets").value;
  const retard = document.getElementById("retard").value;
  const historique = document.getElementById("historique").value;
  const url = `${API_BASE}/api/predire?revenu=${revenu}&prets=${prets}`
    + `&retard=${retard}&historique=${historique}`;
  const reponse = await fetch(url);
  const data = await reponse.json();
  document.getElementById("resultat").textContent =
    `Score predict : ${data.score_predict} (source: ${data.source})`;
}

async function chargerHistorique() {
  const reponse = await fetch(API_BASE + "/api/historique");
  const lignes = await reponse.json();
  const liste = document.getElementById("historiqueListe");
  liste.innerHTML = "";
  lignes.forEach(l => {
    const li = document.createElement("li");
    li.textContent = `revenu ${l.revenu}, ${l.prets} prets, ${l.retard}j retard,
      + `historique ${l.historique} -> ${l.score_predict} (${l.source}`;
    liste.appendChild(li);
  });
}

```

Exercice 1 Construire et démarrer la stack

Containerfile du backend

Besoins à préparer :

- image de base légère (par exemple `python:3.12-slim`) ;
- ordre des instructions optimisé pour le cache de build (dépendances avant les données, données avant le code applicatif) ;
- copie de `data.csv` dans l'image ;
- **entraînement du modèle pendant le build** : exécuter `train_model.py` dans le Containerfile, pour que `model.joblib` et `metrics.json` soient déjà présents au démarrage ;
- exécution sous un utilisateur non-root ;
- instruction `HEALTHCHECK` sur la route `/health`.

Rédigez ce Containerfile, construisez l'image et taguez-la `mtu-ml-backend:1.0`.

Containerfile du frontend

Besoins à préparer :

- image de base légère (par exemple `python:3.12-alpine`) ;
- copie des fichiers `index.html` et `app.js` ;
- démarrage du serveur de fichiers intégré à Python (`python -m http.server`) sur le port 80.

Rédigez ce Containerfile, construisez l'image et taguez-la `mtu-ml-frontend:1.0`.

Réseau et démarrage manuel

1. Créez un réseau Podman dédié.
2. Lancez db (`postgres:16`) avec `db/init.sql` monté pour l'initialisation, et un volume nommé.
3. Lancez cache (`redis:7-alpine`).
4. Lancez backend, avec ses variables d'environnement, en publiant son port vers l'hôte.
5. Lancez frontend, port 8080 publié.
6. Ouvrez `http://localhost:8080`, saisissez des valeurs, cliquez sur "Évaluer", puis sur "Voir l'historique".

Vérifier le cache

1. Appelez deux fois de suite `curl "http://localhost:5000/api/predire?revenu=35000&prets=2&r"` en chronométrant chaque appel.
2. Attendez plus de 60 secondes, puis rappelez la même requête.

Exercice 2

Écrivez un `compose.yaml` avec les services db, cache, backend, frontend.

Besoins à préparer :

- db et cache en images officielles ; backend et frontend construits depuis leurs Containerfile ;
- backend publie son port 5000 vers l'hôte, frontend publie 8080 ;
- volume nommé pour Postgres, réseau implicite ;
- healthcheck sur db et backend, depends_on avec condition: service_healthy.

Validez avec `podman-compose up -d`.

1. Ajoutez une politique de redémarrage automatique à chacun des quatre services.
2. Tuez brutalement le conteneur backend (`podman kill`), puis vérifiez avec `podman ps` qu'il redémarre automatiquement.
3. Dupliquez le service backend en backend2 dans `compose.yaml`, publié sur un port différent de l'hôte (par exemple 5001).
4. Modifiez `app.js` pour alterner les appels entre les deux adresses.
5. Testez en effectuant plusieurs évaluations, en observant séparément les logs de backend et backend2.

Exercice 3 Mettre à jour le modèle et consulter ses performances

Extraire les performances du modèle

1. Interrogez `curl http://localhost:5000/api/performance` et notez les valeurs obtenues (accuracy, precision, recall, f1, tailles des ensembles).

Mettre à jour le modèle par reconstruction d'image (image immuable)

1. Ajoutez ces 3 nouvelles lignes à `data.csv` :
 41000,2,8,Standard,Standard
 59000,0,1,Bon,Eleve
 18000,8,42,Mauvais,Faible
2. Reconstituez l'image backend (`podman build ...`), en réutilisant le même tag `mtu-ml-backend:1.0`.
3. Supprimez l'ancien conteneur backend et relancez-en un nouveau à partir de l'image reconstruite.
4. Interrogez à nouveau `/api/performance`. Les métriques ont-elles changé ?

Mettre à jour le modèle sans reconstruire l'image (volume)

1. Créez, sur l'hôte, un dossier `./data` contenant une copie de `data.csv`.
2. Relancez le conteneur backend en montant ce dossier en bind mount sur `/app/data` (`-v ./data:/app/data:Z`), à la place du précédent.
3. Entraînez le modèle depuis l'extérieur, sans reconstruire l'image ni redémarrer le conteneur : `podman exec backend python train_model.py`.
4. Vérifiez que `model.joblib` et `metrics.json` sont bien apparus dans `./data`, côté hôte.

5. Modifiez directement `data.csv` depuis l'hôte (ajoutez les 3 lignes de la question précédente), relancez `podman exec backend python train_model.py`, puis interrogez à nouveau `/api/performance`.